

Terraform - Complete Interview Prep

Terraform — Complete Interview Prep Guide

For Megha | DevOps Engineer (3-5 Years Experience Level)

1. What is Terraform & Why Use It

“What is Terraform?” > “Terraform is an Infrastructure as Code (IaC) tool by HashiCorp. Instead of manually clicking around the AWS console to create resources, I write code that describes the infrastructure I want — like a VPC, EC2 instances, or an EKS cluster — and Terraform creates, updates, or destroys those resources to match that code.”

“Why use Terraform instead of manually creating resources or using AWS Console/CLI?”
> “Manual changes aren’t repeatable, aren’t version-controlled, and are easy to get wrong across environments. With Terraform, the same code can spin up Dev, QA, and Prod consistently, everything is tracked in Git so I can see who changed what and when, and I can review changes before they’re applied instead of finding out after something breaks.”

“What is the purpose of using Terraform, in one line?” > “To manage infrastructure as version-controlled, repeatable, and reviewable code instead of manual, one-off changes.”

“Is Terraform cloud-specific?” > “No — Terraform is cloud-agnostic. It works with AWS, Azure, GCP, and hundreds of other providers through the same workflow, which is different from something like AWS CloudFormation that only works with AWS.”

2. Core Concepts — Basic to Intermediate

“What is a Terraform provider?” > “A provider is a plugin that lets Terraform talk to a specific platform’s API — like the AWS provider, which lets Terraform create EC2, S3, or EKS resources. Each provider translates my Terraform code into actual API calls to that platform.”

“What is a Terraform resource?” > “A resource is a single infrastructure object I want Terraform to manage — like one EC2 instance, one S3 bucket, or one VPC. It’s defined with a `resource` block specifying the type and its configuration.”

“What’s the difference between `terraform plan` and `terraform apply`?” > “`plan` shows what would change — a preview, without touching anything. `apply` actually makes those changes. I always run `plan` first to review before applying, especially in production, to avoid accidental changes.”

“What is Terraform state, and why does it matter?” > “State is a file (`terraform.tfstate`) where Terraform tracks what infrastructure it’s actually created and its current configuration. It compares this state against my code to figure out what needs to change on the next `plan / apply`. Without state, Terraform wouldn’t know what already exists.”

“Why should state be stored remotely (like in S3) instead of locally?” > “Local state only

lives on one person's machine — if two people run Terraform at the same time, or someone loses their laptop, the state gets out of sync or lost. Remote state (like an S3 backend) is shared, versioned, and can be locked so only one person applies changes at a time, avoiding conflicts."

"What is state locking, and why is it needed?" > "State locking prevents two people from running `terraform apply` at the same time, which could corrupt the state or cause conflicting changes. I've used DynamoDB alongside an S3 backend — S3 stores the state, DynamoDB handles the lock."

"What is a Terraform module?" > "A module is a reusable, self-contained package of Terraform code — like a template for creating a VPC or an EKS cluster. Instead of rewriting the same resource blocks for every environment, I call the module with different input variables for Dev, QA, and Prod."

"What's the difference between a variable and an output in Terraform?" > "A variable is an input — something I pass into a module or configuration, like `instance_type` or `environment`. An output is something Terraform returns after applying — like the VPC ID or a load balancer's DNS name — which other modules or scripts can then use."

3. Advanced Concepts

"What is the Terraform state file's `.tfstate` used for internally, and why shouldn't it be edited manually?" > "It's a JSON file mapping my resource blocks to actual real-world resource IDs. Editing it manually can desync Terraform's understanding from reality, causing it to try to recreate or delete things unexpectedly. If I need to fix state, I use commands like `terraform state rm` or `terraform import`, not a text editor."

"What is `terraform import` used for?" > "It brings existing infrastructure — created manually or by another tool — under Terraform's management, by mapping a real resource to a resource block in my code, without recreating it."

"What's the difference between `count` and `for_each` in Terraform?" > "`count` creates multiple copies of a resource based on a number, indexed 0, 1, 2... `for_each` creates resources from a map or set, indexed by key, which is more stable — if I remove an item from the middle of a list with `count`, it can shift indexes and cause Terraform to destroy/recreate unrelated resources. `for_each` avoids that problem."

"What is a `data` source in Terraform?" > "A data source reads information about existing infrastructure that Terraform doesn't manage — like looking up the latest AMI ID, or an existing VPC's ID — without creating or modifying it."

"What is the difference between `terraform taint` (or `-replace`) and just changing a resource's config?" > "`terraform apply -replace=<resource>` forces Terraform to destroy and recreate a specific resource even if its config hasn't changed — useful if a resource is in a bad state (like a corrupted EC2 instance) and I want a clean rebuild, without needing to change any code."

"How do you manage secrets/sensitive values in Terraform?" > "I mark variables as `sensitive = true` so their values don't print in plan/apply output or logs. For actual secret values, I pull them from AWS Secrets Manager or SSM Parameter Store using a data source at runtime, rather than hardcoding them in `.tfvars` files, which could end up in version control."

"What is a Terraform workspace?" > "Workspaces let you maintain multiple state files for the same configuration — useful for quick environment separation. That said, in practice I've mostly used separate directories or separate `.tfvars` files per environment instead, since workspaces can

get confusing at scale and don't cleanly separate things like different backend configs."

"How do you handle Terraform drift — when the real infrastructure no longer matches the code?" > "Run `terraform plan` regularly (or on a schedule in CI) to detect drift — it shows any differences between actual infrastructure and what's in state/code. If drift was intentional (someone made a manual emergency fix), I'd either update the code to match, or run `terraform apply` to bring it back in line, depending on which is correct."

4. Hands-On Walkthrough (Practice These Commands)

Step 1 — Initialize a project

```
terraform init
```

Downloads the providers and sets up the backend (where state is stored).

Say this: "`init` is always the first command — it downloads the AWS provider plugin and configures the backend, like an S3 bucket for remote state."

Step 2 — Write a basic resource

```
main.tf:
```

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t3.micro"

  tags = {
    Name = "web-server"
  }
}
```

Step 3 — Preview changes

```
terraform plan
```

Output shows `+` for resources to be created, `~` for updates, `-` for deletions.

Say this: "I always read the plan output carefully before applying — especially watching for `-/+` which means destroy and recreate, since that could cause downtime on a live resource."

Step 4 — Apply changes

```
terraform apply
```

Terraform shows the plan again and asks for confirmation (`yes`) before creating resources.

For CI/CD pipelines (non-interactive):

```
terraform apply -auto-approve
```

Say this: “In pipelines, I’d only use `-auto-approve` for lower environments like Dev. For Production, I’d keep a manual approval gate before applying.”

Step 5 — Use variables instead of hardcoding

`variables.tf` :

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "environment" {
  type = string
}
```

`terraform.tfvars` :

```
environment = "dev"
instance_type = "t3.micro"
```

Apply with a specific var file:

```
terraform apply -var-file="prod.tfvars"
```

Say this: “This is how I’d handle Dev vs Prod — same code, different `.tfvars` file per environment, so I’m not duplicating configuration.”

Step 6 — Create and use a module

Folder structure:

```
modules/
  vpc/
    main.tf
    variables.tf
    outputs.tf
environments/
  dev/
    main.tf
```

`environments/dev/main.tf` :

```
module "vpc" {
  source = "../../modules/vpc"
  cidr_block = "10.0.0.0/16"
  environment = "dev"
}
```

Say this: “I’d write the VPC module once, then call it from Dev, QA, and Prod folders with different input variables — this is the same reusability principle as a Helm chart with different values files.”

Step 7 — Check outputs

```
outputs.tf :
```

```
output "vpc_id" {  
  value = aws_vpc.main.id  
}
```

```
terraform output vpc_id
```

Say this: “Outputs let other modules, scripts, or pipelines grab values like a VPC ID or load balancer DNS name after apply, without hardcoding them.”

Step 8 — Set up remote state (S3 + DynamoDB lock)

```
terraform {  
  backend "s3" {  
    bucket      = "my-terraform-state-bucket"  
    key         = "dev/terraform.tfstate"  
    region      = "ap-south-1"  
    dynamodb_table = "terraform-locks"  
    encrypt     = true  
  }  
}
```

Say this: “This is how I’d set up team-safe state — stored in S3, encrypted, with DynamoDB handling locks so two people can’t apply at the same time.”

Step 9 — Import existing infrastructure

```
terraform import aws_s3_bucket.existing my-existing-bucket-name
```

Say this: “If a bucket was created manually before Terraform was introduced, I’d import it so Terraform starts managing it without recreating it.”

Step 10 — Destroy resources safely

```
terraform plan -destroy  
terraform destroy
```

Say this: “I’d always run `plan -destroy` first to review exactly what would be deleted, especially in shared environments, before actually running destroy.”

Step 11 — Format and validate code

```
terraform fmt      # auto-formats code consistently  
terraform validate # checks syntax without hitting any API
```

Say this: “I’d run these in CI before plan/apply — `fmt` keeps code style consistent across the team, and `validate` catches syntax errors early without needing real credentials.”

5. Scenario-Based Questions

“You accidentally ran `terraform apply` on the wrong workspace/environment and it

modified production. What do you do?" > "First, I'd stop and assess — check `terraform plan` output or state history to see exactly what changed. If it's reversible, I'd apply the correct config to revert it, or use `terraform state` commands to fix specific resources. Going forward, I'd add safeguards — separate AWS accounts per environment, or a required approval step in the pipeline before any apply — so this can't happen silently again."

"Terraform state got corrupted or out of sync with real infrastructure. How do you fix it?" > "I'd run `terraform plan` first to see the actual drift. If a resource exists in AWS but not in state, I'd use `terraform import` to bring it back under management. If state references something that no longer exists, I'd use `terraform state rm` to remove just that entry without touching real infrastructure. I wouldn't hand-edit the `.tfstate` file directly unless absolutely necessary."

"Two team members ran `terraform apply` at nearly the same time and it caused a conflict. How do you prevent this going forward?" > "This is exactly what state locking solves — using an S3 backend with a DynamoDB table for locking ensures only one apply can run at a time; the second person's run would wait or fail safely instead of corrupting state. I'd also push for applies to only happen through CI/CD with a single controlled pipeline, rather than everyone running `apply` locally."

"You need to provision the same VPC setup across Dev, QA, and Prod, but each needs slightly different CIDR ranges and instance sizes. How would you structure this?" > "I'd write one reusable module for the VPC, then have separate environment folders (or `.tfvars` files) — `dev.tfvars`, `qa.tfvars`, `prod.tfvars` — each passing in different CIDR blocks and instance types as variables into that same module. This avoids duplicating the actual resource logic three times."

"A resource needs to be replaced (like an EC2 instance in a bad state), but you don't want to change any code. How do you force that?" > "I'd use `terraform apply -replace=\"aws_instance.web\"` — this tells Terraform to destroy and recreate just that resource on the next apply, without needing a code change to trigger it."

"How would you safely roll out a Terraform change that could cause downtime (like resizing a database)?" > "I'd check the plan output carefully for any `-/+` (destroy and recreate) on that resource — some changes, like certain RDS modifications, can be applied in-place, while others force replacement. For anything risky, I'd schedule it during a maintenance window, take a backup/snapshot first, and if possible use features like `create_before_destroy` in a lifecycle block so the new resource is ready before the old one is removed."

"How do you keep Terraform code reviewable and safe in a team CI/CD pipeline?" > "Every change goes through a PR — the pipeline runs `terraform fmt -check`, `terraform validate`, and `terraform plan`, posting the plan output as a PR comment so reviewers can see exactly what will change before approving. `apply` only runs after merge, often with a manual approval gate for production."

"You need to manage secrets like a database password in Terraform without exposing them in code or state. How?" > "I'd avoid putting the actual secret in `.tfvars` or hardcoded values. Instead, I'd pull it from AWS Secrets Manager or SSM Parameter Store using a data source at apply time, and mark the variable as `sensitive = true` so it doesn't get printed in plan/apply logs. Note: the value would still exist in the state file, so the state itself also needs to be encrypted and access-restricted."

6. Quick-Reference Command Cheat Sheet

Task	Command

Initialize project / download providers	<code>terraform init</code>
Preview changes	<code>terraform plan</code>
Preview with a specific var file	<code>terraform plan -var-file="prod.tfvars"</code>
Apply changes	<code>terraform apply</code>
Apply without prompt (CI/CD, lower envs)	<code>terraform apply -auto-approve</code>
Force replace a specific resource	<code>terraform apply -replace="aws_instance.web"</code>
Destroy all resources	<code>terraform destroy</code>
Preview what would be destroyed	<code>terraform plan -destroy</code>
Format code consistently	<code>terraform fmt</code>
Validate syntax	<code>terraform validate</code>
Show current state resources	<code>terraform state list</code>
Remove a resource from state (not real infra)	<code>terraform state rm <resource></code>
Bring existing infra under management	<code>terraform import <resource> <id></code>
Show an output value	<code>terraform output <name></code>
Show full current state	<code>terraform show</code>
Switch/create a workspace	<code>terraform workspace select/new <name></code>

7. The “Structure Your Answer” Trick

Think of Terraform in **3 layers** — if you get a question you haven’t specifically prepped for, place it in one of these and reason from there:

1. **Code layer** — providers, resources, variables, modules → what you write to describe infrastructure
2. **State layer** — state file, locking, drift, import → how Terraform tracks what’s real
3. **Workflow layer** — plan, apply, CI/CD integration, approvals → how changes safely get from code to real infrastructure

8. Closing Line for “Have You Done This Hands-On?”

“Yes — day-to-day I write and maintain Terraform modules for infrastructure like EKS, VPC, and networking, use remote state with S3 and DynamoDB locking for team safety, and every change goes through a plan review in the pipeline before it’s applied — with manual approval gates for production.”

Practice tip: Try `terraform init` and `terraform plan` on a small local project (even just one `aws_instance` resource, without applying it) — seeing real plan output once makes it much easier to describe naturally instead of reciting theory.

